

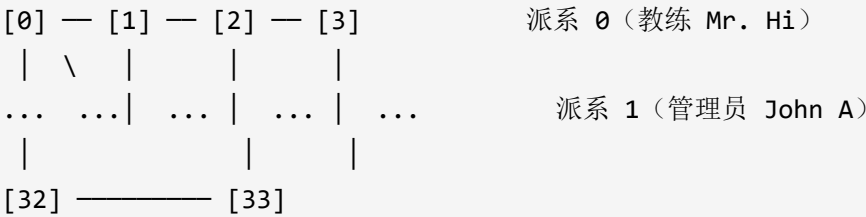


GNN 最简 Demo 解读

1. 整体架构

GNN（图神经网络）处理**图结构数据**——节点通过边相连。核心思想是**消息传递**：每个节点的代表由其邻居的代表聚合而来。

Karate Club 图（34 节点，78 边）：



输入：邻接矩阵 A [34×34] + 节点特征 X [34×34]

输出：每个节点的类别 logits [34×2]

2. GCN 层公式推导

GCN 层的核心公式（Kipf & Welling, 2017）：

$$H' = \sigma(\tilde{D}^{-1/2} \cdot \tilde{A} \cdot \tilde{D}^{-1/2} \cdot H \cdot W)$$

2.1 为什么需要 $\tilde{A}$ (加自环)

$$\tilde{A} = A + I$$

原始邻接矩阵:

[0 1 0 1]

[1 0 1 0]

[0 1 0 0]

[1 0 0 0]

加自环后:

[1 1 0 1]

[1 1 1 0]

[0 1 1 0]

[1 0 0 1]

不加自环, 节点聚合时**会丢失自己的特征**——只能看到邻居。

2.2 为什么需要 $\tilde{D}^{-1/2}$ 归一化

度数差异很大的节点, 聚合后的值差异也很大:

节点 A (度=1): 聚合 1 个邻居 → 值小

节点 B (度=10): 聚合 10 个邻居 → 值大 (可能爆炸)

对称归一化 $\tilde{D}^{-1/2} \cdot A \cdot \tilde{D}^{-1/2}$ 让每行/列的和保持稳定:

对边 (i, j), 归一化权重 = $1 / \sqrt{\deg_i \times \deg_j}$

学位高的节点, 每条边的权重自动降低。

2.3 代码映射

```
class GCNLayer(nn.Module):
    def forward(self, X, A):
        # 1) 加自环
        A_tilde = A + I                    #  $\tilde{A}$ 

        # 2) 对称归一化
        deg = A_tilde.sum(dim=1)           #  $\tilde{D}$  的对角线
        deg_inv_sqrt = deg.pow(-0.5)       #  $\tilde{D}^{-1/2}$ 
        D_inv_sqrt = torch.diag(deg_inv_sqrt)
        A_norm = D_inv_sqrt @ A_tilde @ D_inv_sqrt

        # 3) 线性变换 + 邻居聚合
        support = X @ self.W               #  $H \cdot W$ 
        out = A_norm @ support + self.b     #  $\tilde{A} \cdot H \cdot W + b$ 
        return out
```

3. 消息传递视角

GCN 的每一层可以理解为三个步骤：

对每个节点 v ：

1. 邻居聚合： $h_v = \sum_{u \in N(v)} (1/\sqrt{\deg_v \cdot \deg_u}) \cdot h_u$
2. 线性变换： $h_v = W \cdot h_v$
3. 非线性激活： $h_v = \sigma(h_v)$

示意（节点 θ 的聚合）：

邻居的特征：

$h_1 \rightarrow$ 加权 γ

$h_2 \rightarrow$ 加权 $\vdash$ 求和 $\rightarrow W \cdot \text{sum} \rightarrow \text{ReLU} \rightarrow h_{\theta'}$

$h_4 \rightarrow$ 加权 $\lrcorner$

$h_{\theta} \rightarrow$ 加权 $\lrcorner$ （自己的特征）

层数	每个节点能"看到"的范围
1 层 GCN	直接邻居 (1-hop)
2 层 GCN	邻居的邻居 (2-hop)
K 层 GCN	K-hop 邻域

4. 半监督学习

GNN 经典的使用场景是**半监督节点分类**：

训练集中每类只有 2 个已知标签：
节点 [0, 5] → 派系 0
节点 [32, 33] → 派系 1

测试集：其余 30 个节点

训练时：只对 `train_mask` 内的节点算 `loss`
推理时：前向传播自动把标签信息沿边扩散到全图

这利用了图的**同质性假设**——相连的节点倾向于拥有相同标签。

5. GCN vs CNN vs Transformer

	CNN	Transformer	GCN
数据结构	网格（图像）	序列	图
"邻域"定义	卷积核窗口	自注意力权重	图的边
权重共享	空间平移共享	无（每个位置独立计算）	无，但聚合函数共享
归纳偏置	平移不变性 +	位置编码（显式注入）	图拓扑（显式注入）

	CNN	Transformer	GCN
	局部性		
输入	[B, C, H, W]	[B, seq_len, dim]	[N, dim] + [N, N] Adjacency

6. 运行方式

```
pip install torch
python gnn_demo.py
```

预期输出：

```
Nodes: 34, Edges: 78
Classes: 2, Train nodes: 4, Test nodes: 30
Epoch 1 | Loss: 0.6931 | Test Acc: 53.33%
Epoch 40 | Loss: 0.0200 | Test Acc: 100.00%
Epoch 80 | Loss: 0.0153 | Test Acc: 96.67%
...
Epoch 200 | Loss: 0.0066 | Test Acc: 100.00%

Final Test Accuracy: 100.00%
Correct: 34/34
```

仅用 **4 个**标注节点，GCN 就能正确预测其余 30 个节点——因为图结构本身就包含了大量信息。

7. 关键设计决策

决策	原因
加自环 ($A + I$)	防止节点自己的特征在聚合中丢失
$D^{-1/2}$ 而非 D^{-1}	对称归一化比行归一化更稳定，保留了特征尺度
2 层 GCN	感受野覆盖 2-hop, 太多层会导致过平滑 (所有节点特征趋于相同)
特征 = one-hot 单位阵	无外部特征时，用 one-hot 身份作为初始特征 (每个节点唯一)

8. 从 GCN 到现代 GNN

变体	核心改进
GraphSAGE (2017)	采样邻居 (处理大规模图), 支持多种聚合函数 (mean/max/LSTM)
GAT (2018)	引入注意力机制, 让模型自己学习每条边的权重
GIN (2019)	理论上证明达到 WL 测试的上限 (最强判别力)
RGCN (2018)	处理多种边类型的异构图

此外还有处理**边/图级别**任务的变体 (链接预测、图分类), 以及将 Transformer 应用于图结构的 **Graphormer** 等。

但**消息传递 + 邻居聚合**的核心范式与本 demo 完全一致。